Ex. No. 6	Integrate Animations and Transition
Date of Exercise	12/10/26
GitHub Host Link	https://github.com/jomiwilson/flutter_lab_exp6

Aim

To implement animations and page transitions in Flutter.

Description

This experiment introduces implicit animations in Flutter using the AnimatedContainer widget. A state variable is used to change the width and height of the container when a button is pressed. The AnimatedContainer automatically animates the change in size smoothly over the given duration.

Tools/Software Required: Flutter

Procedure:

Step 1

Create:

```
flutter create animation_app
cd animation_app
code .
```

Step 2 – Create AnimatedContainer

```
AnimatedContainer(
  duration: const Duration(seconds: 1),
  width: 200,
  height: 200,
  decoration: BoxDecoration(
    borderRadius: BorderRadius.circular(20),
  ),
  child: const Center(
    child: Text('Flutter'),
  ),
)
```

Step 3 – Create state variable

```
bool isLarge = false;
```

Step 4 – Change size

```
AnimatedContainer(
  duration: const Duration(seconds: 1),
  width: isLarge ? 300 : 150,
  height: isLarge ? 300 : 150,
  child: const Center(
    child: Text('Flutter'),
```

```
  ),
)
```

Step 5 – Add button

```
ElevatedButton(
onPressed: () {
    setState(() {
     isLarge = !isLarge;
    });
  },
  child: const Text('Animate'),
),
```

Step 6 – Test

Click the Animate button.

Observe the smooth size transition.

Program

lib/main.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: AnimationPage(),
    );
  }
}

class AnimationPage extends StatefulWidget {
  const AnimationPage({super.key});

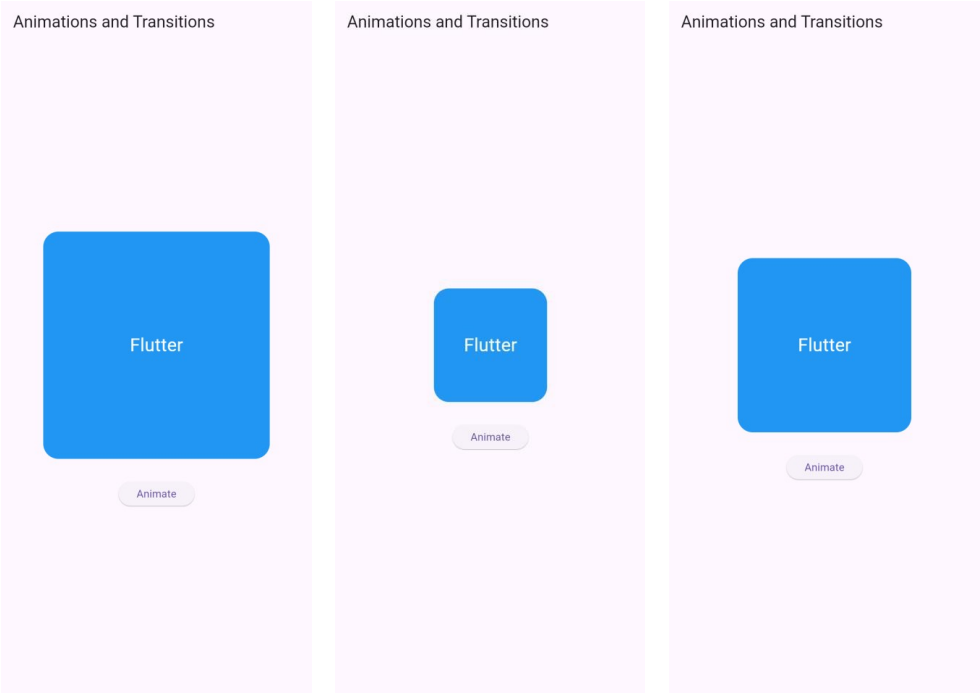
  @override
  State<AnimationPage> createState() => _AnimationPageState();
}

class _AnimationPageState extends State<AnimationPage> {
  // State variable that controls the size of the container
  bool isLarge = false;

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(
        title: const Text('Animations and Transitions'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            AnimatedContainer(
              duration: const Duration(seconds: 1),
              curve: Curves.easeInOut,
              width: isLarge ? 300 : 150,
              height: isLarge ? 300 : 150,
              decoration: BoxDecoration(
                color: Colors.blue,
                borderRadius: BorderRadius.circular(20),
              ),
              child: const Center(
                child: Text(
                  'Flutter',
                  style: TextStyle(color: Colors.white, fontSize: 20),
                ),
              ),
            ),
            const SizedBox(height: 30),
            ElevatedButton(
              onPressed: () {
                setState(() {
                  isLarge = !isLarge;
                });
              },
              child: const Text('Animate'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output



Result

A Flutter animation application was developed successfully. The AnimatedContainer widget displayed a container with rounded corners and text. Pressing the Animate button changed the size of the container using setState. The size change was animated smoothly, demonstrating Flutter animation behavior.